Versa SD-WAN Ansible Automation

1. Overview

The sdwan-automate playbooks communicate exclusively with the Versa Director REST API using HTTPS with HTTP Basic authentication. All request and response bodies are JSON. The base URL is constructed at runtime from the active Director discovered during HA probing:

https://<director_host>:<director_port>/vnms

The default port is 9182. The active Director is identified by probing /vnms/system/isActive on each configured host. All subsequent API calls use the resolved base URL automatically.

Authentication:

Authorization: Basic <base64(user:password)>
Accept: application/json
Content-Type: application/json (on POST / PUT requests)

2. API Call Summary

The table below lists every Director REST call made by the playbooks in execution order.

Step	Method	Path	Purpose
HA Probe	**GET**	/vnms/system/isActive	Discover active Director in HA pair
STEP 2	**GET**	/vnms/sdwan/global/Branch/availableId	Get next available site ID
STEP 3	**GET**	/vnms/sdwan/workflow/devices/device/{name}	Check if device already exists

			and its workflow state
STEP 4	**POST**	/vnms/sdwan/workflow/devices/device/template/data/{name}	Retrieve auto-generated template variable bindings
STEP 6a	**POST**	/vnms/sdwan/workflow/devices/device	Create new device record in Director

Step	Method	Path	Purpose
STEP 6b	**PUT**	/vnms/sdwan/workflow/devices/device/{name}	Update existing device record (variable rebinding)
STEP 7 / STEP A	**POST**	/vnms/sdwan/workflow/devices/device/deploy/{name}	Trigger device deploy / re-deploy
STEP 8 / STEP B / STEP D	**GET**	/vnms/tasks/task/{taskId}?deep=true	Poll async task until terminal state
STEP 9 / STEP E	**GET**	/vnms/sdwan/workflow/devices?deep=true&orgname={org}&offset=0&limit=25	Verify device presence in org
STEP C	**POST**	/vnms/template/applyTemplate/devices	Commit (push) template to live device

3. Director HA Discovery

Runs once per playbook execution (check_director.yml). Probes director_host_1 first, then director_host_2 as fallback. The first host that responds 2XX becomes director_host and director_base_url for the entire run.

GET /vnms/system/isActive — HA probe

Called by	check_director.yml (INIT — before any provisioning step)

Auth	HTTP Basic (same credentials as all other calls)
Body	None
Success	200, 201, or 202 → this Director node is active
Failure	Any non-2XX, timeout, or connection error → try next host
Abort	If both hosts fail: playbook aborts with an error message

Response body: not parsed — only the HTTP status code is checked.

4. STEP 2 — Get Available Site ID

Requests a unique site ID from Director. Director maintains a counter and returns the next free integer. The returned value is stored as site_id and used in the STEP 6 POST body.

GET /vnms/sdwan/global/Branch/availableId — get next site ID

Called by	provision_cpe.yml (STEP 2)
Condition	Always — runs for both new and existing devices
Body	None
Success	200 or 202
Response	A bare integer, e.g. 103

Ansible extraction:

```
site_id: "{{ available_id_result.json | int }}"
```

Note: for existing devices STEP 3 also returns the previously assigned siteId (existing_site_id). STEP 6b (PUT) uses existing_site_id, not site_id, to preserve the original assignment.

5. STEP 3 — Device Status Check

Determines whether the device already exists in Director and, if so, its current workflow state. The response drives the entire branch logic for the rest of the run.

GET /vnms/sdwan/workflow/devices/device/{name} — idempotency check

Called by	provision_cpe.yml (STEP 3)
{name}	device_name — e.g. BRANCH-003-PRIMARY
Body	None
Status codes	200 (found) or 404 (not found) — both are accepted

6.1. Response structure (HTTP 200)


```
{
 "versanms.sdwan-device-workflow": {
 "workflowStatus": "Deployed",
 "applianceExists": true,
 "siteId": 103,
 "postStagingTemplateInfo": {
 "templateData": {
 "device-template-variable": {
 "variable-binding": {
 "attrs": [ ... ]
 }
 }
 }
 }
 }
}
```


6.2. Branch decisions

HTTP	workflowStatus	applianceExists	Action
404	(none)	(none)	New device — full provisioning: STEP 4 POST → STEP 6 POST → STEP 7 deploy → STEP 8 poll → STEP 9 verify
200	Deployed	false	Existing (not live) — rebind vars: STEP 4 (from GET) → STEP 6 PUT → STEP 7 deploy → STEP 8 poll → STEP 9 verify
200	Deployed	true	Live device — update vars then re-deploy: STEP 6 PUT → branch to redeploy_cpe.yml (STEP A+B+C+D+E)

200	Failed	any	ABORT — workflowStatus=Failed; investigate in Director before retrying
200	other	any	ABORT — unexpected workflowStatus; expected Deployed

6. STEP 4 — Template Variable Bindings

Retrieves the full set of template variable bindings (attrs) for the device. For new devices a POST is made to a dedicated template-data endpoint; for existing devices the attrs are extracted directly from the STEP 3 GET response (no extra API call needed).

6.3. New device (POST)

POST /vnms/sdwan/workflow/devices/device/template/data/{name} — fetch template bindings

Called by	provision_cpe.yml (STEP 4) — only when device_exists=false
{name}	device_name
Success	200 or 201

6.4. Request body

```
{
 "versanms.sdwan-device-workflow": {
 "deviceName": "<device_name>",
 "siteId": <site_id>,
 "orgName": "<org_name>",
 "serialNumber": "<serial_number>",
 "deviceGroup": "<device_group>",
 "licensePeriod": <license_period>,
 "deploymentType": "<deployment_type>",
 "modelNumber": "<model_number>",
 "locationInfo": { "country": "<location_country>" },
 "postStagingTemplateInfo": { "templateName": "<template_name>" } }
}
```

6.5. Response — attrs extraction

The attrs array is extracted from the response (same path as STEP 3 for existing devices): response["versanms.sdwan-device-workflow"]
["postStagingTemplateInfo"]["templateData"]
["device-template-variable"]["variable-binding"]["attrs"]

Each element of attrs looks like:

```
{
 "name": "{$v_MPLS_IPv4__staticaddress}",
 "value": "",
 "isAutogeneratable": false,
 "description": "MPLS IPv4 static address"
}
```

isAutogeneratable=true → Director fills the value; no user input needed.
isAutogeneratable=false → user must supply the value via site/CPE vars. STEP 5 aborts if any required value is missing.

6.6. Existing device

For existing devices (device_exists=true, workflowStatus=Deployed) the attrs are taken from device_check_result (the STEP 3 response) using the same path. No extra API call is made.

7. STEP 5 — Variable Merge (no API call)

STEP 5 is pure Ansible logic — no Director API call is made. It:

1. Builds user_supplied_vars: maps each ansible_var from the site/CPE YAML to its corresponding Director variable name ({$v_...}) using vars/templates/<template>.yml.
2. Checks missing_required_vars: any attrs entry with isAutogeneratable=false that has no matching user supplied value causes an abort.
3. Produces final_attrs: the original attrs list with user-supplied values overlaid. This is the payload sent in STEP 6.

Type validation (EARLY VALIDATION block, before STEP 2) also runs with no API call — it checks ipv4 and ipv4_mask format before any HTTP request is made.

8. STEP 6 — Create or Update Device Record

Persists the device record (with merged template variable bindings) to Director. Uses POST for new devices and PUT for existing ones.

8.1. Create — new device (POST)

POST /vnms/sdwan/workflow/devices/device — create device

Called by	provision_cpe.yml (STEP 6a)
Condition	device_exists=false
Success	200 or 201
site_id	Value returned by STEP 2 (newly allocated)

8.2. Update — existing device (PUT)

PUT /vnms/sdwan/workflow/devices/device/{name} — update device

Called by	provision_cpe.yml (STEP 6b)
Condition	device_exists=true AND workflowStatus=Deployed (applianceExists true or false)
Success	200 or 201
site_id	existing_site_id from STEP 3 response (preserves original assignment)

8.3. Request body (same structure for POST and PUT)

```
{
 "versanms.sdwan-device-workflow": {
 "deviceName": "<device_name>",
 "siteId": <site_id>,
 "orgName": "<org_name>",
 "serialNumber": "<serial_number>",
 "deviceGroup": "<device_group>",
 "licensePeriod": <license_period>,
 "deploymentType": "<deployment_type>",
 "modelNumber": "<model_number>",
 "locationInfo": { "country": "<location_country>" },
 "postStagingTemplateInfo": {
 "templateName": "<template_name>",
 "templateData": {
 "device-template-variable": {
 "template": "<template_name>",
 "variable-binding": { "attrs": [ <final_attrs> ] }
 }
 }
 },
 "serviceTemplateInfo": {
 "templateData": {
 "device-template-variable": [{
 "template": "<org_name>-DataStore",
 "organization": "<org_name>",
 "device": "<device_name>",
 "variable-binding": { "attrs": [] },
 "variableMetadata": []
```

```
 }]
 }
 }
 }
}
```

After STEP 6, if applianceExists=true the playbook branches to redeploy_cpe.yml (STEP A) and skips STEP 7-9. Otherwise STEP 7 (deploy) continues.

9. STEP 7 / STEP A — Trigger Deploy or Re-deploy

Submits the device configuration for deployment. Used in two contexts: STEP 7 in provision_cpe.yml (new/not-live devices) and STEP A in redeploy_cpe.yml (live devices). The endpoint and request body are identical.

POST /vnms/sdwan/workflow/devices/device/deploy/{name} — trigger deploy

Called by	provision_cpe.yml STEP 7 (device_exists=false, or exists but not yet live)
Also by	redeploy_cpe.yml STEP A (applianceExists=true)
{name}	device_name
Body	{} (empty JSON object)
Success	200, 201, or 202

9.1 Response

```
{
 "versanms.sdwan-device-workflow": {
 "taskId": "275"
 }
}
```

The wrapper key is variable (Director may use different keys in different firmware versions). The playbook uses a JMESPath wildcard to extract taskId regardless of the wrapper name:
deploy_task_id: "{{ deploy_result.json | json_query('*.taskId | [0]') | default('') }}"

The extracted taskId is passed to STEP 8 / STEP B for polling.

10. STEP 8 / STEP B / STEP D — Poll Async Task

All long-running Director operations are asynchronous. After triggering a deploy or template commit, the playbook polls the task endpoint until a terminal status is reached. The same endpoint is used for all three polling steps.

GET /vnms/tasks/task/{taskId}?deep=true — poll task status

Called by	provision_cpe.yml STEP 8 (deploy poll)
Also by	redeploy_cpe.yml STEP B (re-deploy poll)

Also by	commit_and_verify_cpe.yml STEP D (commit-template poll)
{taskId}	Task ID returned by the triggering POST (STEP 7, STEP A, or STEP C)
Retries	task_poll_retries (default 60) × task_poll_interval seconds (default 10) = 10 min max wait
Success	200

10.1 Response


```
{
 "versa-tasks.task": {
 "versa-tasks.task-status": "SUCCESS",
 "versa-tasks.percentage-completion": 100,
 "versa-tasks.errormessages": null
 }
}
```


10.2 Terminal states

tastask-status	Playbook action
SUCCESS	Continue to next step
COMPLETED	Continue to next step
FAILED	Log error + abort playbook with fail:
ERROR	Log error + abort playbook with fail:
(other)	Retry — poll again after delay

On FAILED or ERROR, the error message from versa-tasks.errormessages is included in the abort message and written to the per-device log.

11. STEP 9 / STEP E — Verify Device in Org

A final GET confirms the device is visible in the organisation after provisioning or commit.

Both the provisioning path (STEP 9) and the redeploy path (STEP E) use the same endpoint.

GET /vnms/sdwan/workflow/devices?deep=true&orgname={org} &offset=0&limit=25 — verify device presence

Called by	provision_cpe.yml STEP 9 (after successful deploy)
Also by	commit_and_verify_cpe.yml STEP E (after successful commit)
{org}	org_name — e.g. NVIDIA-IT
Body	None
Success	200
Assert	device_name appears anywhere in the JSON response body

The playbook uses ansible.builtin.assert rather than uri status_code to perform the check. If the device name is absent from the response, the playbook aborts with a failure message.

12. STEP C — Commit Template to Live Device

Applies (pushes) the template configuration to a live (applianceExists=true) device. This step only runs in commit_and_verify_cpe.yml, which is called from redeploy_cpe.yml after the re-deploy task (STEP B) succeeds.

POST /vnms/template/applyTemplate/devices — commit template

Called by	commit_and_verify_cpe.yml (STEP C)
Condition	appliance_exists=true only
Success	200, 201, or 202

12.1 Request body

```
{
 "versanms.templateRequest": {
 "device-list": [ "<device_name>" ],
 "mode": "overwrite"
 }
```

```
}
```

12.2 Response

```
{
 "versanms.templateResponse": {
 "message": "Apply Template Task was created.",
 "result": true,
 "taskId": "275"
 }
}
```

taskId is extracted with the same JMESPath wildcard used for deploy:

```
commit_task_id: "{{ commit_result.json | json_query('*.taskId | [0]') |  default('') }}"
```

The task is then polled in STEP D using the standard task poll endpoint (section 10), followed by org verification in STEP E (section 11).

13. Error Handling Summary

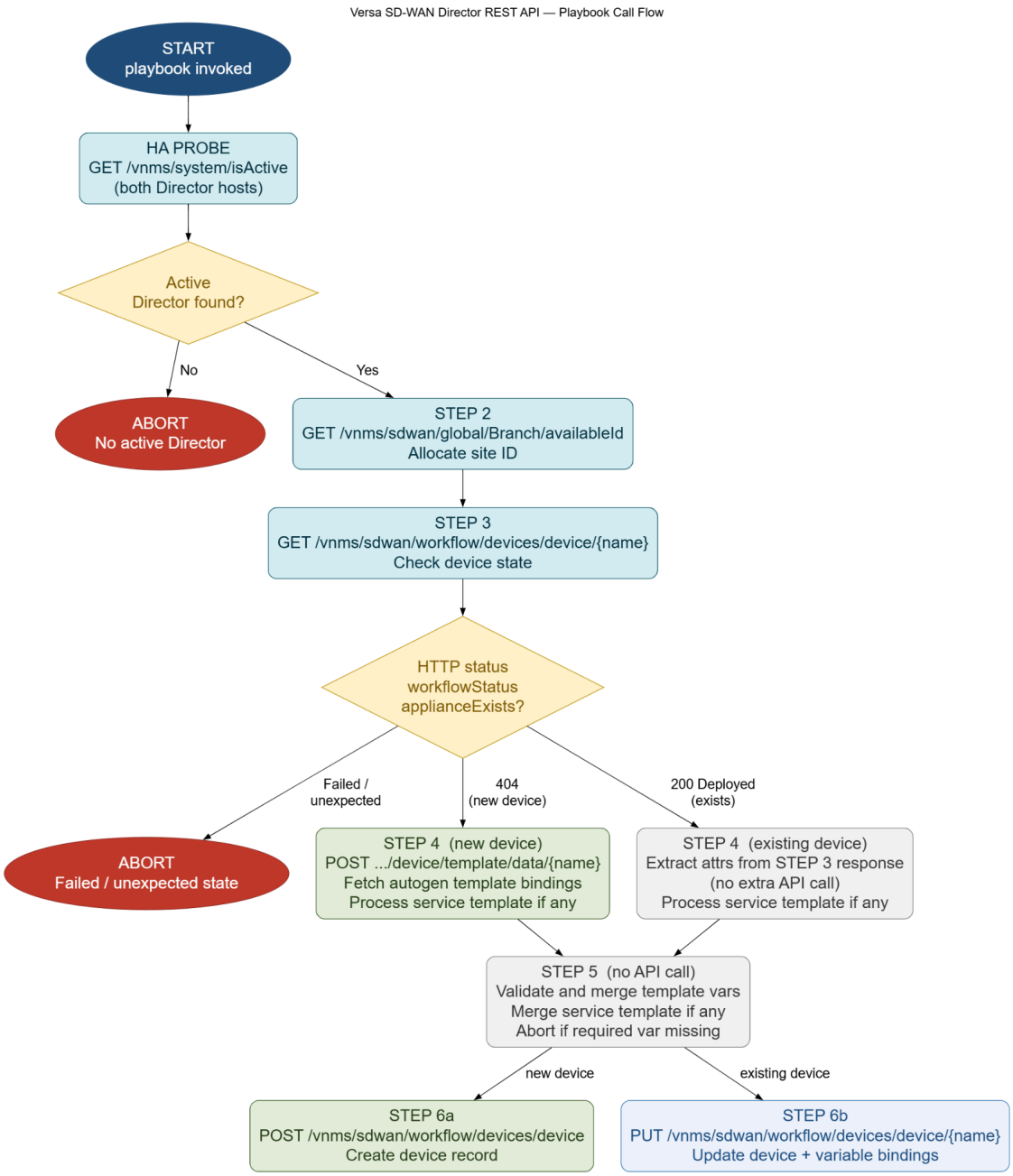
The table below summarises every abort condition across the playbooks.

Step	Trigger	Abort message
HA Probe	Both hosts non-2XX	Neither Director responded as active on /vnms/system/isActive
VALIDATE	Type check fails	N variable(s) failed type validation: <list of ansible_var = value — reason>
STEP 3	workflowStatus=Failed	CPE is in a FAILED state; investigate in Director before retrying
STEP 3	Unexpected workflowStatus	CPE exists but workflowStatus is not Deployed; investigate in Director
STEP 5	Missing required vars	N required template variable(s) have no value; lists each missing director_var and its ansible_var
STEP B	task-status=FAILED/ ERROR	Re-deploy FAILED; includes task status and errormessages
STEP D	task-status=FAILED/ ERROR	Commit template FAILED; includes task status and errormessages

STEP 9 / E	Device absent from org	CPE not found in org after provisioning/commit

14. REST API Call Flow — Workflow Diagram

The diagram below shows the complete Director REST API call sequence across all paths. Color coding: blue-teal = GET, green = POST, blue = PUT, grey = Ansible-only (no API call), yellow=decision point point, red=abort



API call flow continued.